Architecture & High Availability

Scenario: You are designing a microservices system. How do you ensure service failures don't affect entire system?

A: I ensure fault isolation primarily through **Bulkheads** and **Circuit Breakers**. The Bulkhead pattern isolates resources (like thread pools) so one failing service cannot exhaust resources needed by others. The Circuit Breaker pattern monitors service health and quickly stops calling services that have failed, preventing **cascading failures** throughout the system.

What deployment strategies guarantee minimal downtime?

A: Blue-Green Deployment and **Canary Releasing** guarantee minimal downtime. Blue-Green runs two identical environments (old 'Blue' and new 'Green') and instantly switches traffic when 'Green' is validated. Canary releasing exposes the new version only to a small subset of users first, allowing real-world testing before a full rollout.

How do you configure microservices for high availability?

A: High availability is configured by deploying multiple **stateless** instances (replicas) of each service across different availability zones (AZs). I use a **load balancer** to distribute traffic and **Kubernetes** to constantly monitor service health, automatically restarting or replacing any failed instances.

How would you architect multi-region microservices deployment?

A: I would architect a multi-region deployment using an **Active-Active** or **Active-Passive** strategy. Identical services are deployed in two or more regions. Global DNS (like Route53) routes users to the nearest healthy region. Data must be replicated between regions using a multi-region database (e.g., global Aurora or Cassandra) for resilience.

Distributed Data & Saga Patterns (Deep)

How do you handle distributed transactions when dozens of services interact?

A: I handle this using the **Saga Pattern**, which breaks the global transaction into a sequence of local transactions in different services. Each local transaction publishes an event to trigger the next step. This avoids the heavy locking and complexity of traditional two-phase commit (2PC).

How do you ensure global consistency during failures?

A: Global consistency is ensured via **eventual consistency**. I use the **Transactional Outbox Pattern** to reliably publish events. If a Saga step fails, **compensation actions** are triggered—

these are dedicated transactions designed to explicitly undo the changes made by previously successful services, ensuring state integrity.

Saga compensation logic with real examples.

A: Compensation logic is the reversal of a successful action. For a checkout Saga, if Inventory service deducts stock but the Payment service fails, the Inventory service executes a compensation action: it consumes the "Payment Failed" event and **restores (adds back)** the stock it previously deducted.

Event sourcing + CQRS trade-offs.

A: Event Sourcing (ES) stores every state change as a sequence of immutable events, ensuring 100% data fidelity but increasing complexity. **CQRS (Command Query Responsibility Segregation)** separates the read model (optimized for queries) from the write model (optimized for commands). The trade-off is high **complexity** and delayed consistency (read-after-write) in exchange for massive **read scalability**.

Advanced Communication & Scaling

How do you secure communication between hundreds of microservices?

A: I secure communication using **Mutual TLS (mTLS)**, where every service verifies the cryptographic certificate of the service it is talking to. This is often automated using a **Service Mesh** (like Istio), which handles certificate rotation and encryption/decryption transparently.

How do you design asynchronous event-driven systems at scale?

A: I design using a **Message Broker** (Kafka) as the central backbone. The core design principles are **stateless services**, **event schema governance** (using a schema registry), and **consumer partitioning** to allow parallel processing of messages by multiple consumer instances.

When do you choose event streaming vs REST vs gRPC?

A:

- **REST/HTTP:** Choose for **synchronous** request-response (e.g., retrieving a user profile) where immediate feedback is needed.
- **gRPC:** Choose for **synchronous** internal communication where low latency and high bandwidth are required (using HTTP/2 and optimized serialization).
- **Event Streaming (Kafka):** Choose for **asynchronous** communication, notification, and data pipelines where loose coupling and resilience are critical.

API Gateway at Scale

Explain the role of an API Gateway in a large-scale ecosystem.

A: The API Gateway is the single entry point for all client traffic. Its role is to handle **traffic routing** (mapping public URLs to private service endpoints), **protocol translation**, and **load balancing**. It simplifies the client's interaction with the complex, private internal network.

What functionalities improve security at the gateway?

A: Security is improved by centralizing **Authentication** (validating user credentials or tokens), **Authorization** (checking initial access rights), and **SSL/TLS Termination** (decrypting HTTPS traffic). The gateway acts as a security filter before traffic hits the backend services.

How to implement rate-limiting, caching, token validation at the gateway level.

A: Rate-limiting is implemented using a global filter that checks a token count against a central store (like Redis) for each client/IP. **Token validation** is done via a filter that verifies the JWT signature and expiration. **Caching** is implemented to store responses for non-volatile read operations, reducing load on downstream services.

Observability at Enterprise Scale

What distributed tracing architecture would you implement?

A: I would implement a tracing architecture based on the **OpenTelemetry (OTEL)** standard. Every service is instrumented to generate traces and spans. These traces are collected by an OTEL agent and exported to a central backend (like **Jaeger** or **Zipkin**) for visualization.

How to trace across 20–40 microservices?

A: You trace across services by ensuring every service propagates the **Trace ID** and **Span ID** in the request headers (B3 or W3C context). This mechanism stitches together all the individual log entries and metrics from the 20-40 services into one unified timeline.

What logging & monitoring architecture would you choose?

A: I would choose the **ELK Stack** (Elasticsearch, Logstash/Fluentd, Kibana) or **Loki/Prometheus** for a unified observability platform. **Logging** is structured (JSON format) and centralized. **Metrics** are collected via Prometheus scraping endpoints on every service.

How would you design dashboards?

A: I would design dashboards using the **Golden Signals** approach: focusing on **Latency** (response times), **Traffic** (requests/sec), **Errors** (rate of failure), and **Saturation** (CPU/Memory utilization). This provides an immediate, clear view of the system's health.

Service Mesh Architecture

What is service mesh and why use it?

A: A **Service Mesh** is a dedicated infrastructure layer that handles service-to-service communication transparently, typically implemented using sidecar proxies (like Envoy). You use it to offload complex, cross-cutting concerns—like **mTLS encryption, retries, traffic routing, and advanced metrics**—from the application code.

How does service mesh handle retries, mTLS, load balancing, traffic shaping?

A: The service mesh handles these functions automatically via the **sidecar proxy** (Envoy). The sidecar intercepts all incoming/outgoing traffic: it performs **mTLS** encryption/decryption, executes smart **retries** and **load balancing** to healthy instances, and performs **traffic shaping** (e.g., routing 5% of traffic to a new version).

Istio/Linkerd advantages vs API Gateway-only design.

A: The primary advantage is **L7 (application layer) visibility and control** over **internal** service communication, not just edge traffic. An API Gateway only secures the perimeter. Istio/Linkerd enforce **mTLS**, dynamic routing (traffic splitting), and monitoring between *all* microservices, creating a truly zero-trust network.

Infrastructure & Cloud Strategy

Docker vs Kubernetes vs ECS/EKS vs serverless: when to choose what?

A:

- **Docker:** Essential for **packaging** all microservices (always required).
- **Kubernetes (EKS/AKS):** Choose for **complex, large-scale systems** requiring fine-grained control over networking and scaling.
- **ECS/Azure Container Apps:** Choose for simpler, managed container orchestration with less operational overhead than raw K8s.
- **Serverless (Lambda/Azure Functions):** Choose for **event-driven functions** with unpredictable traffic and low runtime cost requirements.

How do you design autoscaling policies for microservices?

A: I design autoscaling policies using **Horizontal Pod Autoscaling (HPA)** in Kubernetes. Policies are based on both **CPU utilization** (scale up when CPU hits 70%) and **application metrics** (scale up when the Kafka consumer lag or HTTP request queue depth exceeds a threshold). I also define a minimum and maximum number of replicas for resource stability.

How do you design CI/CD for 50+ microservices?

A: I design CI/CD using independent, fast, parallel pipelines. Each microservice has its own build and deployment pipeline that runs only when its code changes. I enforce **Contract Testing (Pact)** at the build stage to ensure component compatibility. Deployment is automated using **Kubernetes** and **Helm** for packaging, enabling fast, isolated rolling updates.